During the scene, the actor does a `soliloquy` and a `stunt`—both methods of the `actor` object. A double that takes the place of the actor during the scene without ruining it needs to know how to do (or pretend to do) things that the actor has to do. Create a double that can `soliloquy` and `stunt` using an object literal:

```
actorDouble =
  soliloquy: ->
  stunt: ->
```

The scene completes successfully with this `actorDouble` in place of the `actor`:

```
scene = (actor) ->
  actor.soliloquy()
  actor.stunt()
  'Scene completed'

fact 'The scene is completed', ->
  actorDouble =
    soliloquy: ->
    stunt: ->
  assert.equal scene(actorDouble), 'Scene completed'
# The scene is completed: OK
```

This technique of providing dependencies to part of a program is called *dependency injection*. Creating doubles in this way to pass them in as arguments is straightforward, if at times a little inconvenient, in CoffeeScript.

Consider the `extractData` function that contains a dependency on `http`. Any tests against it will suffer the same difficulties that the tests for `scene` had: